LAB 08 / WEEK 04

MLOps: Machine Learning Operations

Notice. Explain. Recover.

Service Monitoring, Incident Diagnosis & Rollback Verification.
Continue your team's Bike Demand service on Cloud Run, with Lab 7 reviewed on main and versions retained.

OBSERVE



DIAGNOSE



RECOVER

Done means a recovery claim with evidence

Three outcomes you can show from your own running service.

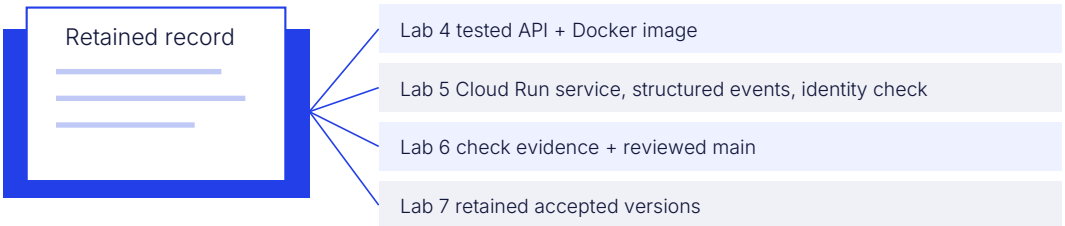


Notice a real change, explain the evidence and restore a retained accepted state on Cloud Run.
The record should let a reviewer reconstruct the conclusion.

A signal you missed is an honest result to improve.

Start from the system you already built

This is the continuation of Labs 4–7.



Keep the tested API and image, structured events and independent version check. Locate the current and previous accepted versions, their integrity records and the reviewed gate decisions.

Use your README; missing prerequisites remain explicit blockers.

Rotate the operator, injector and recorder

The operator should diagnose from evidence.



Operator

Investigate without the answer



Injector

Keep one fault reversible



Recorder

Preserve actual evidence



The injector chooses one reversible fault inside the isolated rehearsal. The operator investigates; the recorder keeps the timeline. Every member owns or substantively reviews work.

Agree roles and scope first; reveal the fault after the diagnosis record.

Leave a record another person can find

Only these paths are fixed; your internal design is yours.

WHERE EVIDENCE LIVES

- RUNBOOK.md: at repo root
- reports/: record name is yours
- README.md: links to evidence
- Team report: Lab 8 section



Another operator follows the evidence.

Add the runbook and incident record through reviewed work. Link the actual raw evidence from the README and team report so a reviewer can follow the chain.

Screenshots support raw evidence; keep at most three.

Preserve the API and its meaning

Existing clients and validation rules must keep working.

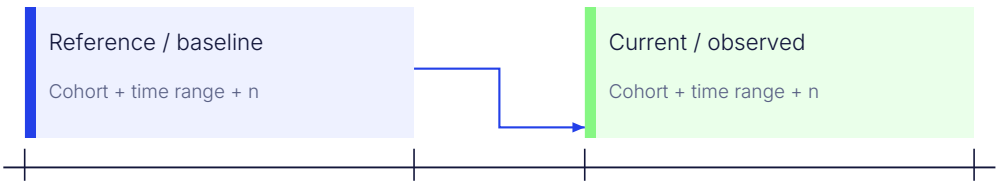
1	Readiness	Usable loaded model
2	Prediction	Finite nonnegative value + version
3	Rejection	Invalid request refused

Keep GET /health (200 when usable, otherwise 503) and POST /predict (prediction plus the loaded model_version). Invalid input still receives 4xx; valid input without a usable model receives 503.

No training inside the service. No new mandatory /version endpoint.

Define normal before the incident

Your baseline must come from an actual observed run.



same definitions; explain relevant time and cohort differences

Decide which signals would reveal a meaningful change. Record workload, environment, identity, observation window and count before the injector acts.

A chart without its population and window is incomplete evidence.

Give each signal a clear definition

Keywords guide the design; justify your choices.

Good eligible attempts

All eligible attempts

SAME POPULATION / SAME WINDOW

- Eligibility
- Units and window
- Count and missing data

Explain eligibility, numerator, denominator, units, window and missing-data behavior. Distinguish intentional validation rejections from failed valid predictions.

Choose and justify your policy; no universal target is given.

Find the first signal before naming the cause

Save the observation that started the investigation.

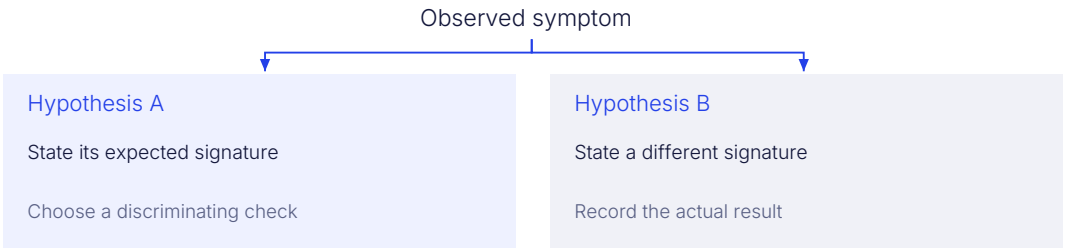
1	Observed signal	Exact time and affected scope
2	Evidence	Raw event or external result
3	Interpretation	A hypothesis to test

Record the signal, its UTC time, affected scope and the version or revision observed. Link the raw event, external result or aggregate that supports it.

Do not rewrite raw evidence to match the final explanation.

Test at least two plausible explanations

A symptom is evidence; a cause is a conclusion to justify.

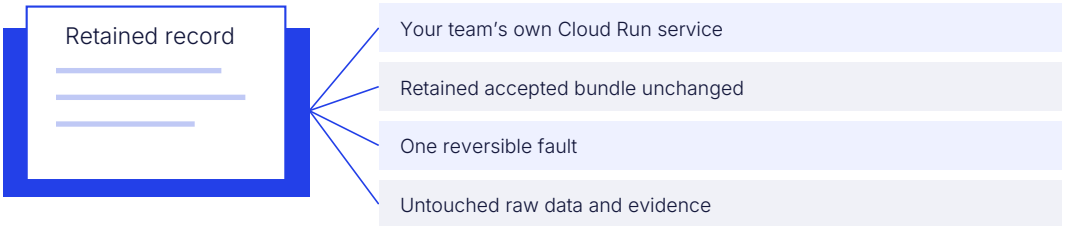


For each hypothesis, state its predicted observation, the check you chose, the actual result and what remains uncertain. A plausible story alone is insufficient.

Keep failed checks and rejected explanations in the incident record.

Keep the incident isolated and reversible

One controlled change should be enough to test the monitoring loop.



Inject into your own team’s Cloud Run service only; a local rehearsal first is fine. Pick a release or artifact fault a retained compatible bundle can reverse. Preserve accepted artifacts and raw data.

Never touch another team’s Cloud Run service; every fault stays reversible.

Ask what changed at each boundary

Use the cheapest informative checks first.

Service	Revision and image
Model	Version and verified bytes
Configuration	Effective settings and schema
Inputs	Behavior and dependencies

Compare service revision, image, model identity, configuration and incoming-data behavior. State which observations support the diagnosis and which boundaries you have not checked.

A mismatch is evidence; explain whether it causes the observed symptom.

Choose the response your evidence supports

Explain the tradeoff before claiming a fix.



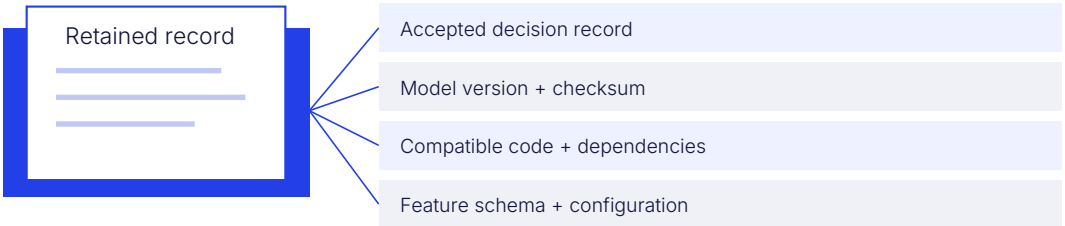
Evidence	Action	Limits
What failed and why?	Why this mitigation?	What could still fail?

Explain why rollback fits the core fault, then restore the retained accepted state. Guards and reviewed forward fixes are valid options for other causes; investigate those as an optional extension after the core drill.

Recover to a retained version on Cloud Run; do not retrain as the recovery action.

Name the rollback target before switching

A target is a compatible retained bundle, not “whatever was previous.”



Identify the accepted version and integrity record. Check compatibility with the service, dependencies, feature schema, configuration and the retained Cloud Run revision before making one controlled change.

A checksum mismatch must fail closed; do not load or silently relabel that artifact.

Verify after the recovery action

Fresh observations must support the recovery claim.

1	Identity	Retained expectation vs observed state
2	Behavior	Fresh requests to the live Cloud Run URL
3	Signals	Post-change events, Cloud Run logs and window

Use independently retained identity expectations, external behavior and post-change signals. State the observation window and explain how the three sources agree.

A successful command or stopped alert alone does not prove recovery.

Compare baseline, incident and recovery

Keep the measurement definition stable across the three periods.

Baseline	Incident	After mitigation
Count + scope Identity + signals Your actual results	Count + scope Identity + signals Your actual results	Count + scope Identity + signals Your actual results

For each period, retain scope, count, identity and observed signals. Explain what improved, what remains unknown and what evidence could falsify the recovery claim.

Use actual results. No expected success values are pre-filled.

Keep every conclusion proportional to its evidence

A blocker or a missed signal is part of the result.

1	Observed	What actually happened
2	Unknown	What has not been established
3	Next action	Owner and concrete blocker

Record the failing step, exact message, current state and next owner. Separate local execution from remote CI, and service health from predictive quality. Report a Cloud Run access or quota blocker; drill locally.

Never edit logs, weaken checks or invent a passing run.

Have a peer use the runbook

The reviewer should be able to act without the author narrating.



Write symptoms, first checks, compatible recovery target, safe action and verification. Add escalation, limitations and the last verified context; revise after the peer attempt.

The incident record tells what happened; the runbook helps the next operator.

Preserve the fixed evaluation boundary

The incident drill does not open the held test period.

TRAIN	VALIDATION	HELD TEST
2011	Jan–Jun 2012	Jul–Dec 2012

Use permitted validation-period observations for replay and any quality comparison. The service still never trains, and labels stay outside predictor inputs.

No test-period metric is computed in Weeks 1–4.

Review the complete operational story

The team checkpoint joins the pipeline gate and the incident record.



Lab 7	Lab 8	Review
Gate decisions + retained bundle	Incident + verified recovery	Coherent evidence on main

Link gate decisions and retained versions from Lab 7 to diagnosis, rollback and recovery evidence from Lab 8. Review changes before merging to main; promotion remains a human decision.

Passing CI or a quality gate does not automatically deploy.

Submit the checkpoint and your own account

One team checkpoint; one graded individual report.

TEAM

m4

Ungraded team checkpoint

INDIVIDUAL

reports/week-4.md

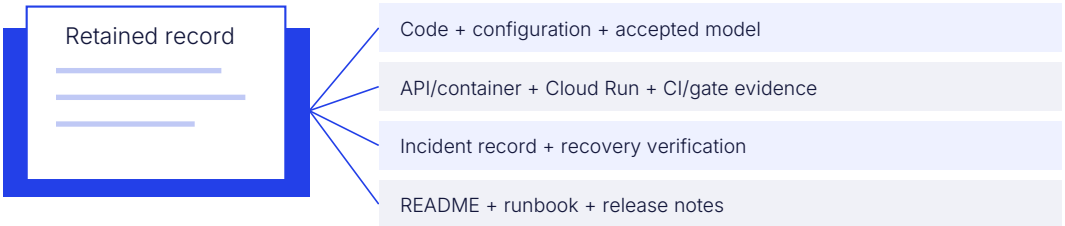
10% of course grade

After Labs 7–8 are reviewed, submit an annotated m4 tag link and a short note. Each student explains their contribution, decisions, evidence, problems and assistance in the personal report.

Individual report: due Sunday at 23:59 in the published time zone. m4 is ungraded.

Prepare the final engineering handover

Week 5 remains the project clinic, verification and presentations.



Prepare a GitHub Release v1.0.0 linking code, configuration, accepted model, checks, incident evidence, README, runbook and release notes. Explain remaining limitations honestly.

Team project and presentation: 60%. Week 5: clinic, verification, presentations; no individual report.

After Lab 8: the minimum before presenting

Lab 8 ends the new content. This is the minimum that remains.

1	Now	Submit m4 and your own reports/week-4.md
2	Before Week 5	Choose your release candidate; fix the gaps your evidence shows
3	Week 5	Clinic, final verification, then the team presentation

Week 5 adds no new lab and no individual report. Release v1.0.0 and the presentation are assessed.
The next slide lists what to show.

The held test is scored once, on the candidate you already chose; never to pick between models.

Week 5: what you present and submit

Show seven things, each backed by evidence in your repository.

	SHOW	EVIDENCE
1	System and purpose	README and one architecture sketch
2	Reproducible run	Clean checkout, lock file and Docker image
3	API and container tests	pytest results and CI runs
4	Gate decisions	Rules, an accept and a reject, with reasons
5	Logging and incident	Events, first signal, RUNBOOK.md, verified recovery
6	Final held-test result	One evaluation of your selected candidate
7	Limits	What the system and the gate cannot tell you

Release v1.0.0 links code, configuration, model, evidence, README, runbook and release notes.

Each member answers questions on their own work. Your instructor announces dates, order and time.

LAB 08 / EXIT CHECK

Would the next operator recover faster?

Show the first signal, the explanation you tested, the retained version you restored, and the fresh evidence that supports recovery.